

RemSolution — Complete Roadmap v2.1 (Consolidated)

July 13, 2026 · Supersedes v2 — adds the accepted items from the external review (RowVersion, selective soft delete, file metadata, Hangfire, AgencySettings, pricing seam, outbox, API versioning, ...) plus final ops items. This is the single source of truth.

Multi-tenant SaaS for rental agencies + public Airbnb-style marketplace. .NET 10 / Angular / SQL Server, Clean Architecture, single database shared schema with AgencyId global query filters.

Status assumption: Phases 0, 0.5 and 1 are complete as originally specified. Amendments defined after those phases shipped appear as **Backfill** tasks — still to do.

Legend: ✓ **Done** · ■ **Backfill** (amendment to a completed phase) · ■ **To do**

Business model (locked)

- **Platform admin (one, seeded):** countries, brands, models; creates agencies + their admin; assigns subscription plans (MaxCars, MaxUsers, MaxClients); enables feature modules per agency (ExtraServices, Expenses, Statistics); read-only cross-tenant visibility via a dedicated audited path.
- **Agency admin:** cars assigned to Branches (pickup locations with geography); agency users + per-user CRUD permissions; contract/invoice templates; ExtraServiceType and ExpenseType (threshold by months OR mileage) if features enabled; statistics if enabled. Holds all permissions implicitly.
- **Agency user:** clients, rentals, reservations, payments, expenses, extra services — each action gated by agency feature flag AND user permission, enforced at the API.
- **Marketplace client:** picks country/place + dates → available cars from nearby branches → books (forced account: email + phone; uploads passport + driving licence) → reservation PendingConfirmation (holds the car, with expiry) → agency confirms → payment → pickup/conversion → afterwards: in-app messaging, rating (car/agency, anchored to completed renting), reporting.
- **Mobile (later):** single role-based app (decision pending); agency scans client documents (OCR) → auto-fills the contract. Only requirement today: store document originals at full resolution.

Phase 0 — Hygiene ✓ Done (2–3 days)

#	Task	Status
0.1	Retarget to .NET 10 LTS, bump packages, rebuild, re-run tests	✓ Done
0.2	Squash EF migration (drop Todo tables, remove PendingModelChangesWarning suppression)	✓ Done
0.3	Delete Domain.User; ApplicationUser as single identity source; domain references users by string UserId	✓ Done
0.4	Externalize connection string (user secrets / Key Vault)	✓ Done
0.5	Delete template-leftover domain events + PriorityLevel enum	✓ Done
0.6	Drop Bootstrap, keep Angular Material only	✓ Done
0.7	Update commands for Brand, ModelCar, Country; seed countries	✓ Done

Phase 0.5 — Multi-tenancy, subscriptions & logging ✓ Done + ■ Backfill (4–5 days done, +2.5–3 days backfill)

#	Task	Status
T.1	Agency root entity, NetTopologySuite enabled, platform-admin CRUD	✓ Done
T.2	AgencyId on every tenant table; tenant-scoped indexes lead with AgencyId	✓ Done
T.3	ITenantProvider from JWT claim; global query filters; SaveChanges interceptor stamps AgencyId; IgnoreQueryFilters() restricted to MarketplaceSearch + audited platform-admin path	✓ Done

#	Task	Status
T.4	SubscriptionPlan (MaxCars, MaxClients) + AgencySubscription; limits enforced under sp_getapplock; expired → block writes	✓ Done
T.5	Roles: PlatformAdministrator / AgencyAdministrator / AgencyStaff; AgencyId claim; seeded platform admin	✓ Done
T.6	Serilog structured logging (AgencyId/UserId/CorrelationId enrichers) + AuditLog table via [Auditable] pipeline behaviour	✓ Done
B.1	Branch entity — Branch(Id, AgencyId, Name, CountryId, Location geography), CRUD for agency admin, spatial index on Location, IX_Branch(AgencyId, ...). Geographic anchor moves from Agency to Branch.	■ Backfill 0.5 d
B.2	Permission system replaces fixed AgencyStaff role — AgencyFeature(AgencyId, Feature, Enabled) + UserPermission(UserId, Permission) in JWT, enforced by policies + [RequiresFeature(...)] pipeline behaviour. Feature off ⇒ 403 at API and module hidden in Angular. AgencyAdministrator = all permissions implicitly. Tables + plumbing now; admin UI later.	■ Backfill 1 d
B.3	SubscriptionPlan.MaxUsers + enforcement in CreateAgencyUser under the same sp_getapplock discipline	■ Backfill 0.3 d
B.4	Add platform-admin cross-tenant reads explicitly to the [Auditable] list	■ Backfill 0.1 d
B.7	JWT lifetime strategy — permissions/features live in the JWT, so revocation must work: short-lived access tokens (10–15 min) + refresh token that re-reads permissions and feature flags on refresh. No version-stamp machinery.	■ Backfill 0.5 d

Phase 1 — Client vertical slice ✓ Done + ■ Backfill (1 week done, +1 day backfill)

#	Task	Status
1.1	Client CRUD commands/queries, vertical-slice layout, Mapster projections, nullable MarketplaceUserId reserved	✓ Done
1.2	FluentValidation: CIN / Passport / Driving Licence blocks, dates, Country FKs	✓ Done
1.3	IFileStorage abstraction; paths in SQL + files in blob/wwwroot	✓ Done
1.4	DeleteBehavior.Restrict on multiple Country FKs	✓ Done
1.5	Angular Client CRUD (Material table, forms, image upload), NSwag regenerated	✓ Done
B.5	Rich file metadata — StoredFile record: Path, OriginalFileName, MimeType, Size, SHA-256 hash, UploadedBy/At, DocumentType tag (Passport, DrivingLicence, CIN, CarPhoto, ...). Originals kept at full resolution (mobile OCR later). Hash supports dedup + the PII/legal trail (7.5).	■ Backfill 0.5 d
B.6	Client.IsFlagged + Client.Notes (per-agency bad-client flag; cross-agency blacklist deliberately excluded). No generic polymorphic EntityNote table — plain columns where needed, FK integrity preserved.	■ Backfill 0.2 d

Pre-Phase-2 gate ■ To do (blocking decisions + schema + cross-cutting plumbing, ~3.5–4 days)

Everything here touches the Car schema, the Money model, or a cross-cutting pattern every later slice inherits — must land before the Renting aggregate exists.

#	Task	Detail	Effort
P.1	Car.BranchId FK	Every car belongs to a pickup Branch (requires B.1). Availability and marketplace search anchor on Branch.	0.2 d
P.2	Car.Status	Active / Maintenance / Inactive. Filter in every availability query. No separate CarStatusHistory table — mark the status-change command [Auditable] and AuditLog is the history.	0.2 d

#	Task	Detail	Effort
P.3	Car .DailyRate + pricing seam	Flat daily rate v1. All price calculation routed through ONE domain service: IPricingService.Quote(car, dates, extras) → Money. Flat implementation reads DailyRate; seasonal/weekend/promo pricing later swaps the internals of one seam — no PricingPolicy table hierarchy now (speculative). Rule: after booking, NEVER read Car.DailyRate again — the price snapshot on Reservation/Renting is the contract price.	0.5 d
P.4	Money value object with currency	Amount + currency code + rounding. Precision decided: decimal(18,2) everywhere (no currency conversion, no fractional-rate math). v1: one currency per agency (in AgencySettings, P.9).	0.5 d
P.5	UTC decision	All dates stored UTC (or DateTimeOffset), converted at the edge. Decided and documented before the first StartDate is written.	0.1 d
P.6	Car photos + thumbnail pipeline	CarImage(CarId, StoredFileId, SortOrder, IsPrimary). On upload generate thumbnail + medium (original kept) — resizing at request time doesn't survive marketplace traffic. Runs as a Hangfire job (P.10).	0.5 d
P.7	Car form update	Angular: branch picker, status, daily rate, photo upload with previews.	0.5 d
P.8	RowVersion optimistic concurrency	byte[] RowVersion (IsRowVersion()) on Car, Client, Renting, Reservation, Agency via base configuration. DbUpdateConcurrencyException → 409 with a clear error code; Angular surfaces "reloaded by another user". Prevents silent lost updates between agency staff.	0.3 d
P.9	AgencySettings entity	One owned entity/table instead of ALTER TABLE per setting: CurrencyCode, CancellationWindowHours, ReservationExpiryHours (+ future settings). Read through a cached provider.	0.3 d
P.10	Hangfire adopted now	The 3.2 scheduler decision made early: Hangfire (SQL Server storage) is the single job infrastructure — reservation expiry, thumbnails, emails, PII purge, statistics refresh all become one-liners instead of per-feature hosted services. Dashboard secured platform-admin only.	0.5 d
P.11	Selective soft delete	NOT blanket. Car + Client: IsDeleted/DeletedAt/DeletedBy (= archived, history preserved); ExpenseType/ExtraServiceType: IsActive flag (deactivation, not deletion); Renting/Payment/Reservation: NEVER deleted by anyone — financial records. Global filter composes: !IsDeleted && AgencyId == tenant. Unique indexes become filtered (WHERE IsDeleted = 0) or a deleted client's CIN can never be reused. FKs stay Restrict — physical deletes of referenced rows already fail.	0.7 d
P.12	Collation decision	Accent-insensitive collation (e.g. Latin1_General_100_CI_AI) on name/search columns NOW — French/Arabic client names. Changing collation on populated columns later rebuilds every index touching them.	0.2 d

Phase 2 — Renting core ■ To do (2–2.5 weeks)

#	Task	Detail	Effort
2.1	Renting as rich aggregate	State machine (NotYet/InProgress/Done) in the domain: private setters, factory Renting.Create(...) enforcing invariants (end > start, price ≥ 0), Start() / Complete(endMileage) guarding transitions, RentingCompletedEvent → RentingHistory snapshot. Handlers: load → call → save. Money VO from P.4; price via IPricingService (P.3). Domain events stay in-process/in-transaction for now (interceptor dispatch) — the outbox arrives with the first out-of-process consumer (6.4).	1.5 d
2.2	CreateRenting + race-safe availability	sp_getapplock keyed on CarId inside one transaction around overlap check + insert. Overlap: newStart < existingEnd AND newEnd > existingStart; decide same-day turnaround. Predicate covers Renting AND Reservation in non-terminal states from day one. Exclude Car.Status ≠ Active and soft-deleted cars.	3 d
2.3	Return / close flow	CompleteRentingCommand: end mileage ≥ start, transition to Done, final price incl. extra services (via IPricingService), RentingHistory snapshot.	2 d
2.4	ExtraServices on renting	Attach/detach lines; ExtraServiceType CRUD (agency admin, behind ExtraServices feature flag, IsActive not delete). Included in price.	1 d

#	Task	Detail	Effort
2.5	Queries + availability indexes	GetRentingById, paginated filtered list (standardized PagedRequest: Page, PageSize, Sort, Search — reused by every later list), car-availability query. Indexes: IX_Renting(CarId, StartDate, EndDate) INCLUDE (State) + mirror on Reservation.	1.5 d
2.6	Tests — DDD pyramid, as you build	Domain: Create rejects invalid dates/price; Complete() from NotYet throws; mileage + event; Money arithmetic/currency mismatch. Application: validators; handler orchestration. Functional (Testcontainers + Respawn): overlap incl. boundaries; two parallel CreateRenting → exactly one succeeds; RentingHistory written; subscription limit; feature flag off → 403; missing permission → 403; concurrent edit → 409 (RowVersion); soft-deleted car invisible through the filter but restorable.	2.5 d
2.7	Vehicle condition at pickup/return	Condition notes + optional photos on Start() and Complete() (StoredFile). Feeds dispute handling (7.3).	0.5 d
2.8	Angular Renting UI	List + detail, create wizard, return dialog with condition capture; 409 conflict handling pattern established here. NSwag regen.	2 d

Phase 3 — Reservation + Payment ■ To do (1.5–2 weeks)

#	Task	Detail	Effort
3.1	Reservation as rich aggregate (state machine)	PendingConfirmation → Confirmed → Paid → Converted, plus Rejected / Expired / Cancelled — factory, guarded transitions, domain events. ExpiresAt + reason fields: RejectedReason (required — the client sees why), CancelledReason, ExpiredReason (analytics later). Price snapshot at booking (P.3 rule). Documents attach to the Reservation.	2 d
3.2	Expiry background job	Hangfire recurring job (P.10): auto-expire unconfirmed reservations after AgencySettings.ReservationExpiryHours.	0.3 d
3.3	Convert reservation → renting	Allowed only from Confirmed/Paid; same per-car lock; carries dates/price/advance payment. Client dedup decided here: on conversion, match existing Client by CIN/passport number within the agency → link (fill MarketplaceUserId) instead of blindly creating a duplicate row; no match → create. Documents move onto the Client.	1.2 d
3.4	Payment recording	Payments against client / renting / reservation. Money VO, decimal(18, 2) explicit. Invariant PayedPrice vs Price decided + enforced. Client balance query. Refund flag. Provider integration out of v1; the Paid state exists.	1.5 d
3.5	Deposit & cancellation policy	DepositAmount on Renting/Reservation; cancellation window from AgencySettings checked in CancelReservation. Business rule decided before validators are written.	0.5 d
3.6	Angular UI	Reservation calendar/list with states + confirm/reject (with reason)/convert actions, payment entry, client account view.	1.5 d

Phase 4 — Expense & maintenance notifications ■ To do (1 week) — cuttable

#	Task	Detail	Effort
4.1	Expense + ExpenseType CRUD	Per-car expenses (amount, date, mileage). ExpenseType: ThresholdKind (Months / Mileage) + value — exclusive choice, domain invariant. IsActive, not delete. Behind the Expenses feature flag.	1.5 d
4.2	'Due' dashboard query	Cars whose last expense of a type exceeds its threshold (branches on ThresholdKind). No push — dashboard query only.	1.5 d
4.3	Angular UI	Expense entry per car, overdue-maintenance dashboard widget.	1 d

Phase 5 — Hardening & ops ■ To do (~1.5 weeks) — keep 5.3/5.5/5.6 even if cut

#	Task	Detail	Effort
5.1	Backfill tests by DDD layer	Domain: Reservation state machine incl. expiry + reasons, Payment/Money edges, Expense thresholds, pricing service. Application: every validator, [Auditable] + [RequiresFeature] behaviours. Functional: conversion carries advance payment + dedup matching; marketplace booking under concurrency.	2 d
5.2	One E2E acceptance path	SpecFlow + Playwright, domain language: create client → rent → return → pay. One reliable E2E.	1.5 d
5.3	Authorization pass	Policies from permission claims + feature-flag behaviour across all endpoints; refresh-token flow (B.7) verified: disabled feature/permission gone within one token lifetime. Functional tests: tenant isolation (404), permission denial, feature-flag denial.	1 d
5.4	Seed + demo dataset	Countries, brands/models, branches, cars with photos, clients — plus the Phase 6 load-test dataset generator.	0.5 d
5.5	Backup & restore rehearsal	Automated SQL backups (+ blob storage for files) AND one rehearsed restore into a clean environment, documented. Multi-tenant: one lost DB is every customer. Not a someday item.	0.5 d
5.6	CI/CD + deploy discipline	Pipeline: build → tests (domain/application on PR, functional nightly or on main) → deploy. EF migrations applied at deploy time via script/bundle (dotnet ef migrations bundle), NOT Migrate() at app startup (multi-instance race). Health check endpoint (AddHealthChecks + SQL check) wired to the host's probe.	1 d

Phase 6 — Public marketplace ■ To do (3–4 weeks)

#	Task	Detail	Effort
6.1	Marketplace user self-registration	Public Identity flow, separate from agency staff; email + phone required; email verification. Consent capture: T&C/privacy acceptance + timestamp stored on MarketplaceUser — legally required before holding passports. One global account; Client row per agency at conversion (3.3).	2 d
6.2	Cross-tenant availability search	Read-only, anonymous, the ONLY IgnoreQueryFilters() besides the audited admin path. Branch filter by CountryId + STDistance (spatial index), join Car (Active, not deleted), NOT EXISTS anti-joins on Renting + non-terminal Reservation, order by distance. Lean DTO: car, daily rate, primary photo (thumbnail), agency, branch, distance, rating average.	3 d
6.3	Marketplace booking flow	Public booking → PendingConfirmation under the same per-car sp_getapplock + document upload + expiry. Idempotency key on CreateReservation : client-generated GUID, unique index — double-click/mobile retry returns the existing reservation instead of creating a second one (the lock serializes, it does not dedupe). Agency confirms/rejects, converts via 3.3. v1 payment on pickup.	3 d
6.4	Notifications + outbox	The flow is dead without notifications — and this is the first out-of-process side effect, so the outbox pattern lands here : domain/integration events written to an Outbox table in the same transaction, a Hangfire dispatcher delivers (email via IEmailSender, in-app notification rows). Commit-then-crash loses nothing. Events: reservation created (→ agency), confirmed/rejected/expired (→ client), payment recorded. Webhooks later ride the same outbox.	1.5 d
6.5	Public endpoint hardening	Rate limiting (AddRateLimiter) on search + registration; CAPTCHA or verified email on signup; upload constraints (size, content-type sniffing, extension whitelist); documents served only through an authorized endpoint — never public blob URLs.	1 d
6.6	Public API conventions	/api/v1/ prefix on marketplace endpoints (versioning is cheap now); machine-readable error codes via ProblemDetails (CAR_UNAVAILABLE, RESERVATION_EXPIRED, CONCURRENT_UPDATE) so Angular localizes; public slugs (/agency/best-rent-tunis, /car/{slug}) — slug column + unique index, generated from name.	1 d

#	Task	Detail	Effort
6.7	Public Angular frontend	Country/place picker → map/list of nearby available cars (photos, rates, ratings) → detail gallery → booking with account creation + document upload → client account area. Angular i18n for the public area (FR/EN at minimum) using the 6.6 error codes; agency-entered data (brands, types) NOT translated — proper nouns and agency-language text. Separate routing area; anonymous until booking.	4.5 d
6.8	Search-path load sanity check	Seeded dataset at target scale (2,000 agencies / 100k cars / 1M+ renting rows); execution plan: spatial seek on Branch, index seeks on the anti-joins. Caching / denormalized read model only if traffic demands — additive under CQRS.	1 d

Phase 7 — Post-rental, communication & documents ■ To do (1.5–2 weeks)

#	Task	Detail	Effort
7.1	In-app messaging	Conversation(ReservationId/RentingId, AgencyId, MarketplaceUserId) + Message. Only after a confirmed reservation. Polling first; SignalR only if needed.	2 d
7.2	Ratings	Review(RentingId unique, Target: Car / Agency, Stars, Comment). Only the marketplace user of a completed renting, once. Averages surfaced in 6.2.	1 d
7.3	Reports (signalement)	Report(ReporterId, Target, Reason, Status); platform admin triage screen — audited cross-tenant write.	1 d
7.4	Contract & invoice generation	Agency-uploaded docx template + placeholder mapping, filled on RentingCompleted via a Hangfire job off the outbox. OpenXML or QuestPDF — decide then.	2.5 d
7.5	PII retention & erasure (legal)	Retention policy (purge documents N months after last renting — Hangfire job), right-to-erasure path (anonymize Client, purge files by StoredFile hash trail, keep financial rows). Policy decided now, implemented here.	1 d
7.6	Statistics module	Agency dashboard (revenue, utilization, top cars/clients) behind the Statistics feature flag; platform-admin cross-tenant aggregates. Query-side; event-fed read models only if these queries prove slow at real volume.	2 d

Deliberately rejected / deferred (so they stop resurfacing)

- **Blanket soft delete** — selective only (P.11); financial records are never deleted by anyone.
- **CarStatusHistory table** — AuditLog + [Auditable] on the status command already is the history.
- **Generic polymorphic EntityNote** — no FK integrity, hole in the tenant-filter guarantee. Plain columns.
- **PricingPolicy entity hierarchy** — the IPricingService seam (P.3) buys the same option for free.
- **Outbox in Phase 2** — zero out-of-process consumers until 6.4; in-transaction dispatch is safe until then.
- **Data localization** (translating brands/types) — proper nouns and agency-language text. UI i18n only (6.7).
- **Resumable uploads (tus)** — phone photos are a few MB; size limits + server-side compression suffice.
- **Version-stamp JWT invalidation** — short-lived access + refresh (B.7) is the simpler correct fix.
- **Online payment provider, seasonal pricing, mobile apps + OCR, GPS, cross-agency blacklist, push notifications, search read model, webhooks** — unchanged from v2; webhooks and read models now have a ready seam (outbox, CQRS) when wanted.

Timeline summary

Phase	Scope	Duration	Status	Cuttable?
0	Hygiene	2–3 d	✓ Done	—
0.5	Tenancy, subscriptions, logging	4–5 d	✓ Done	—

Phase	Scope	Duration	Status	Cuttable?
—	Backfill B.1–B.7	3–3.5 d	■ Backfill	No — B.1/B.2 block Phase 2
1	Client slice	1 wk	✓ Done	—
—	Pre-Phase-2 gate P.1–P.12	3.5–4 d	■ To do	No — schema + cross-cutting patterns
2	Renting core	2–2.5 wk	■ To do	No
3	Reservation + Payment (state machine)	1.5–2 wk	■ To do	No — marketplace books via Reservation
4	Expense & notifications	1 wk	■ To do	Yes
5	Hardening & ops	1.5 wk	■ To do	Partially — keep 5.3, 5.5, 5.6
6	Public marketplace	3–4 wk	■ To do	Deferable, not cuttable — the product goal
7	Post-rental, communication, documents	1.5–2 wk	■ To do	7.4/7.5 no; 7.1–7.3/7.6 deferable

Remaining work: ~12–14 weeks. (v2 was 10–12; the review adoptions add ~1.5–2 weeks, concentrated in the gate, Phase 5 ops, and Phase 6.)

Immediate next steps, in order: B.1 (Branch) → B.2 (permissions) → B.3/B.4/B.7 → B.5/B.6 → P.1–P.12 (one migration for the schema items) → start 2.1.

Definition of done: a marketplace client picks a country, sees available cars with photos and rates from nearby branches for their dates, books idempotently, gets confirmed, notified and pays; the agency completes rent → return → pay with an auto-generated contract — with functional tests covering overlap logic, state transitions, subscription limits, permissions/feature flags, concurrency (lock + RowVersion), soft-delete visibility, and tenant isolation; backed up, restorable, and deployed through CI/CD.